

8. Gün Notlarım: Canlı Veri Nasıl Akıyor?

Dün mesajın nasıl oluşturulduğunu öğrenmiştim. Bugünkü soru farklıydı: bu mesajları saniyede defalarca üretilip, aynı anda birden fazla kişiye canlı olarak nasıl gösterebilirim? Yani bir nevi "gerçek zamanlı yayın" yapmayı öğrendim.

Sürekli çalışan bir döngü kurmak

30 aracın her birinin hızını sürekli güncelleyen ve her 100 milisaniyede bir yeni mesaj üreten bir döngü yazdım. Burada önemli olan şey, bu döngünün diğer işleri (mesela birinin API'ye istek atmasını) bloklamaması gerektiği. Bunun için `asyncio` denen, Python'un "aynı anda birden fazla işi bekletmeden yürütme" yöntemini kullandım. Yani araba A'nın hızını hesaplarken araba B'nin mesajı da gecikmeden gönderiliyor.

Ayrıca öğrendiğim bir şey: bir araç "hatta değil" (offline) durumuna alınırsa, o araç için hiç mesaj üretmiyorum. Bu, gerçek hayatta motoru kapalı bir aracın CAN hattına hiç veri basmamasıyla aynı mantık.

Hız nasıl gerçekçi değişiyor?

Hızı bir anda 0'dan 100'e sıçratmak istemedim, çünkü gerçek bir araç öyle davranmaz. Bunun yerine bir "ivme" mantığı kurdum: hedef hıza doğru saniyede belli bir miktar (örneğin 5 km/h) artarak yaklaşıyor. Fren basıldığında bu azalma daha hızlı oluyor (örneğin saniyede 14 km/h). Ayrıca gerçek bir tekerlek sensörü gibi görünmesi için hıza ufak, rastgele bir "gürültü" (noise) de ekledim — yani hız asla tam düz bir çizgi gibi durmuyor, hafif titriyor.

Aynı anda birden fazla kişiye veri göndermek: WebSocket

Normal bir web isteğinde (REST) tarayıcı sunucudan bir şey ister, sunucu cevap verir ve bağlantı kapanır. Ama benim burada istediğim şey sürekli, kesintisiz bir akıştı — sunucunun kendiliğinden "işte yeni veri geldi" diye tarayıcıya göndermesi. Bunun için WebSocket kullandım.

WebSocket'i öğrenirken en çok şu noktayı anladım: bir bağlantı açıldığı an önce tüm filonun anlık durumunu (snapshot) gönderiyorum, sonra her 100 ms'de bir sadece "ne değişti" bilgisini gönderiyorum. Böylece her seferinde her şeyi baştan göndermek zorunda kalmıyorum, bant genişliğinden tasarruf ediyorum.

Bir diğer önemli nokta: her bağlı istemci için ayrı bir kuyruk (queue) tuttum.

Sebebi şu — eğer bir kullanıcının interneti yavaşsa, o kişi yüzünden diğer herkesin veri akışı yavaşlamamalı. Herkesin kendi kuyruğu olunca, yavaş biri sadece kendi

kuyruğunda birikme yaşıyor, başkasını etkilemiyor.

Komutları hem REST hem WebSocket üzerinden verebilmek

Kullanıcı bir aracın hızını değiştirmek istediğinde bunu iki şekilde yapabiliyor:

normal bir REST isteğiyle (`POST /api/vehicles/.../speed`) ya da WebSocket üzerinden bir mesaj göndererek. İkisinin de arkada aynı komutu tetiklemesini istedim, bu yüzden komut işleme mantığını tek bir yerde topladım ki iki yol da aynı davranışı üretsinsin.

Arayüz tarafında öğrendiğim performans dersi

30 aracı saniyede 10 kere güncellerken, ekrandaki HTML elemanlarını her seferinde yeniden oluşturmak (yani DOM'u sil-baştan kurmak) tarayıcıyı çok yoruyor ve arayüz takılıyor. Öğrendiğim çözüm şuydu: kartları ve satırları sadece bir kere oluştur, sonrasında sadece içindeki sayıları güncelle. Ayrıca ekran güncellemelerini `requestAnimationFrame` denen bir tarayıcı fonksiyonuyla birleştirdim — bu da tarayıcının kendi çizim ritmine uygun şekilde güncelleme yapmamı sağlıyor, gereksiz yere fazla çizim yapmamamı sağlıyor.

Arayüzde neler var

Kısaca özetlersem, ekranda şunlar bulunuyor:

- Marka marka gruplanmış araç kartları, her kartta anlık hız ve bağlantı durumu.
- Hızı elle değiştirebileceğim bir gösterge ve kaydırıcı (slider).
- Gönderilen ham CAN mesajının hangi byte'ının hıza karşılık geldiğini gösteren bir

görsel döküm — yani teoriyi (dünkü SPN 84 anlatımını) gözle görülür hale getiren kısım.

- Zaman damgalı, canlı akan bir mesaj kaydı (log) — istersem duraklatabiliyorum.